20 June 2026



Università degli Studi di Padova

Università Degli Studi di Padova

Cyber-Physical Systems and IoT Security 2025/2026

Alessandro Tigani Sava, student ID: 2198854

Second project report

Authentication of IoT Device and IoT Server Using Secure Vaults

1 Objectives

The scope of this project is to replicate the existing work that identifies limitations in single-password authentication systems by introducing a multi-key mutual authentication mechanism. The core of this approach is a "secure vault", which is a collection of shared keys that functions as the secret between the IoT device and the server.
These vault contents are refreshed following each successful communication session based on the data shared between the server and the device.

The primary objective of the project is to implement the authentication mechanism described in the reference paper and measure the time required to complete the authentication process, these measurements will then be compared against the results reported in the original study.
To accomplish this, it is required to simulate both an IoT device and a server to facilitate the authentication workflow.
Lastly, following the evaluation method of the original paper, the multi-key mechanism will be compared with an ECC-based (Elliptic Curve Cryptography) authentication scheme to evaluate its relative performance.

2 Setup

While the original study implemented this mechanism using an Arduino device, this project recreates the system using local Python code.
To execute the implementation, it is recommended to establish a virtual environment and install the dependencies listed in requirements.txt.
This can be achieved by running the following commands:

```
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

The main dependencies are:

- ecdsa;
- pycryptodome

The project folder contains a single file which execute both the simulations for the secure vault mechanism and the ECC. To execute the simulation is succifent to run the following code:

```
py main.py
```

3 Experiments

3.1 Description

For simplicity, the code is structured as a single file containing the classes required to run the simulation.
The classes included are as follows:

- **Vault**: representing a collection of shared keys that functions as the secret between the IoT device and the server;

- **Server**: responsible for device authentication and vault synchronization, capable of using both the secure vault mechanism and the ECC for authentication;
- **DeviceSVAuth**: rapresenting a device requesting an authentication using a secure vault mechanism;
- **DeviceECCAuth**: representing a device requesting an authentication using the ECC mechanism.

The code consists of two main phases, the first simulates the authentication of multiple devices using a secure vault, while the second performs the authentication of a new device using ECC (Elliptic Curve Cryptography).
In the first phase, after defining parameters such as the number of keys to insert into the vault and the number of devices to simulate concurrently, the Server and SecureVault objects are instantiated.
A number of devices, as specified in the initial configuration, are then created and launched on separate threads to simulate real multi-device communication.
The script waits for each thread to complete, and if authentication is successful, it displays the averages of the performance evaluation using the following metrics:

- Encryption Time.
- Decryption Time.
- Vault Update Time.
- Total Time.

In the second phase, a new device is created that interacts with the same server.
At the end of the operations, the following metrics are collected:

- Key Generation Time.
- Sign Time.
- Verify Time.
- Total Time.

3.2 Classes

SecureVault	
Parameter	**Type**
id	int
keys	list[bytes]
Method	**Return type**
__init__(number_of_keys, key_size)	
perform_xor_over_keys(index_list)	bytes
update_keys	

Table 1: SecureVault class

The SecureVault class in Table 1 implements a cryptographic key storage and management system designed for multi-key authentication schemes.

Server	
Parameter	**Type**
id	int
c1_lengtht	int
key_size	int
lock	threading.Lock
registered_devices_ECC	dict
registered_devices_vault	list
vault	SecureVault
Method	**Return type**
__init__(vault, id, c1_length, key_size)	
register_device_secure_vault(device_id)	
register_device_ECC(device_id, public_key)	
generate_challenge_1(device_id)	Tuple[list, bytes]
validate_challenge_1(device_id, encrypted_response)	bool
generate_final_response(device_id)	bytes
synchronize_vault(device_id)	
is_signature_valid_ECC(device_id, key, signature)	bool

Table 2: Server class

It maintains a collection of symmetric keys that are dynamically updated after each communication session.

The Server class in Table 2 is the component responsible for managing device authentication, handling challenge-response protocols, and synchronizing the shared cryptographic vault. It supports both the proposed Secure Vault mechanism and traditional ECC-based authentication side-by-side.
In the case of authentication using the secure vault, it employs a locking mechanism on the vault until the process is complete, preventing errors if multiple authentications occur simultaneously.
It creates the first authentication challenge by sampling random vault key indices and generating a server nonce, then decrypts the device's response using the derived key, verifies the match, and extracts the second challenge parameters.
As a final response it computes a dynamic session key, encrypts the final response, and prepares the payload for the device.
A the end it updates the global SecureVault keys using session data to ensure forward secrecy after successful authentication.
In case of an ECC authentication mechanism it is also able to verify the digital signature.

The DeviceSVAuth class in Table 3 represents an IoT device client implementing the Secure Vault authentication protocol.
As a threaded component, it handles the complete authentication flow with the server while collecting detailed performance metrics for benchmarking purposes.
It first register itself with the server before initiating the authentication process, then after receiving the first challenge it derives key from selected vault indices, generates temporary tokens and challenge parameters, then encrypts payload using AES-ECB.
The device sends the encrypted response to server and waits for final response containing r2.
After receiving the final response it computes the dynamic session key, decrypts the server response, and verifies r2 match.

DeviceSVAuth	
Parameter	**Type**
device_id	int
server	Server
key_size	int
vault	SecureVault
c2_length	int
authentication	dict
Method	**Return type**
__init__(device_id, server, c2_length, key_size, vault)	
run()	

Table 3: DeviceSVAuth class

If the authentication process is successful it triggers the server-side vault update.
During all the steps of the authentication process it measure the time required at each step.

DeviceECCAuth	
Parameter	**Type**
device_id	int
server	Server
private_key	int
public_key	int
authentication	dict
Method	**Return type**
__init__(device_id, server)	
get_public_key()	bytes
sign_key(key)	bytes
run()	

Table 4: DeviceECCAuth class

The DeviceECCAuth in Table 4 class implements a standard Elliptic Curve Cryptography (ECC) authentication client as a threaded component.
It serves as the control benchmark to compare against the Secure Vault mechanism, performing key generation, digital signing, and server-side verification.

4 Results

The original paper bases its analysis on two parameters:

- execution time;
- energy consumption.

In the current project, direct energy consumption measurements were not feasible, therefore only the execution time of the two analyzed algorithms was evaluated, comparing the results with the data reported in '`Table 1: Energy Consumption of Different Protocols`' from the original study to assess goal achievement.
It should be noted that compared to the initial Arduino-based prototype, the SecureVault

implementation demonstrated significantly faster speeds during execution.
It is save to say that this performance difference can derive from the substantially different hardware specifications between the environment described in the research and the PC used for local simulations.
Table 5 illustrates the results obtained in terms of execution time.

Step	**Time (ms)**
Encryption	0.2688
Decryption	0.2796
Vault Update	0.0760
Total	0.5644

Table 5: Secure Vault AVG Execution Time

Similarly, the ECC-based benchmark recorded lower execution times compared to reference values, for reasons attributable to the explanations given above.
The complete execution data are available in Table 6.

Step	**Time (ms)**
Key Generation	0.041
Sign	0.6301
Verify	2.2297
Total	2.8639

Table 6: ECC Execution Time

Even if the hardware utilized is different, it is possible to confirm that the proposed Secure Vault mechanism outperforms ECC-based authentication in terms of overall speed.